

Universidad Católica de Temuco

Facultad de Ingeniería

Proyecto Matemática Discreta

Teoría de Grafos Aplicada

Integrantes: Danna Ramírez
Matías Manquela
Amanda Contreras
Vicente Fonseca

Profesor: Elliot Mardones Arias

Asignatura: Matemática Discreta

Entrega: 29 de junio de 2026

Contents

1	Introducción	2
2	Modelamiento Matemático	2
2.1	Definición del Grafo	2
2.2	Conjunto de Vértices	2
2.3	Conjunto de Aristas y Función de Peso	2
2.4	Costo de un Camino	4
2.5	Ruta Óptima	4
2.6	Propiedad de Conexidad	4
2.7	Propiedades Formales del Grafo	4
3	Metodología	5
3.1	Algoritmo Seleccionado	5
3.2	Funcionamiento General	5
3.3	Pseudocódigo	6
3.4	Comparación con Algoritmo Alternativo: Bellman-Ford	7
4	Implementación	7
4.1	Estructura General de la Aplicación	7
4.2	Representación del Grafo	7
4.3	Implementación del Algoritmo de Dijkstra	8
4.4	Librerías Utilizadas	9
4.5	Interfaz Gráfica	9
5	Resultados	9
5.1	Prueba 1: Roma $\rightarrow$ Múnich	10
5.2	Prueba 2: Viena $\rightarrow$ Bari	10
5.3	Prueba 3: Niza $\rightarrow$ Roma	10
6	Conclusiones	11

1. Introducción

La búsqueda de rutas óptimas es un problema ampliamente estudiado en matemáticas, informática y logística. Determinar el camino más conveniente entre dos puntos permite optimizar recursos, reducir costos y mejorar la eficiencia de sistemas de transporte, distribución y navegación.

En este proyecto se modeló una red de ciudades europeas mediante un grafo ponderado. Cada ciudad fue representada como un vértice y cada conexión entre ciudades como una arista. Como criterio de ponderación se utilizó la **distancia aproximada entre ciudades expresada en kilómetros**, tomando como referencia distancias reales por vía terrestre obtenidas mediante Google Maps (2026).

El conjunto de ciudades seleccionado incluye **Roma, Nápoles, Perugia, Pescara, Florencia, Bari, Pisa, Bolonia, Génova, Venecia, Milán, Niza, Zúrich, Múnich y Viena**. Estas ciudades fueron escogidas debido a la existencia de conexiones terrestres que permiten construir un grafo conexo y representativo de una red de transporte real.

El objetivo principal del proyecto es diseñar e implementar una aplicación capaz de calcular la ruta de menor distancia entre una ciudad de origen y una ciudad de destino, utilizando teoría de grafos y el algoritmo de Dijkstra como método de resolución.

2. Modelamiento Matemático

2.1. Definición del Grafo

La red de ciudades se representa mediante un **grafo ponderado no dirigido** definido como:

$$G = (V, E, w)$$

donde V es el conjunto de vértices (ciudades), E es el conjunto de aristas (conexiones) y $w : E \rightarrow \mathbb{R}^+$ es la función de peso que asigna a cada arista una distancia positiva expresada en kilómetros.

2.2. Conjunto de Vértices

$$V = \{\text{Roma, Nápoles, Perugia, Pescara, Florencia, Bari, Pisa, Bolonia, Génova, Venecia, Milán, Niza, Zúrich, Múnich, Viena}\}$$

$$|V| = 15$$

2.3. Conjunto de Aristas y Función de Peso

Las conexiones entre ciudades, junto con sus pesos en kilómetros, se presentan en la Tabla 1. Las distancias fueron obtenidas de Google Maps (2026) y corresponden a recorridos por vía terrestre.

Table 1: Conjunto de aristas E con función de peso w (km por vía terrestre)

N°	Arista (u, v)	$w(u, v)$ (km)	Fuente
1	Roma – Nápoles	226	Google Maps (2026)
2	Roma – Perugia	173	Google Maps (2026)
3	Roma – Pescara	208	Google Maps (2026)
4	Roma – Florencia	275	Google Maps (2026)
5	Nápoles – Bari	261	Google Maps (2026)
6	Perugia – Florencia	151	Google Maps (2026)
7	Pescara – Bari	310	Google Maps (2026)
8	Pescara – Perugia	251	Google Maps (2026)
9	Florencia – Pisa	84	Google Maps (2026)
10	Florencia – Bolonia	105	Google Maps (2026)
11	Pisa – Génova	163	Google Maps (2026)
12	Bolonia – Venecia	161	Google Maps (2026)
13	Bolonia – Milán	214	Google Maps (2026)
14	Génova – Milán	142	Google Maps (2026)
15	Génova – Niza	194	Google Maps (2026)
16	Milán – Venecia	278	Google Maps (2026)
17	Milán – Zúrich	279	Google Maps (2026)
18	Venecia – Viena	582	Google Maps (2026)
19	Zúrich – Múnich	310	Google Maps (2026)
20	Múnich – Viena	401	Google Maps (2026)

Por lo tanto $|E| = 20$.

La estructura completa del grafo G se muestra en la Figura 1.

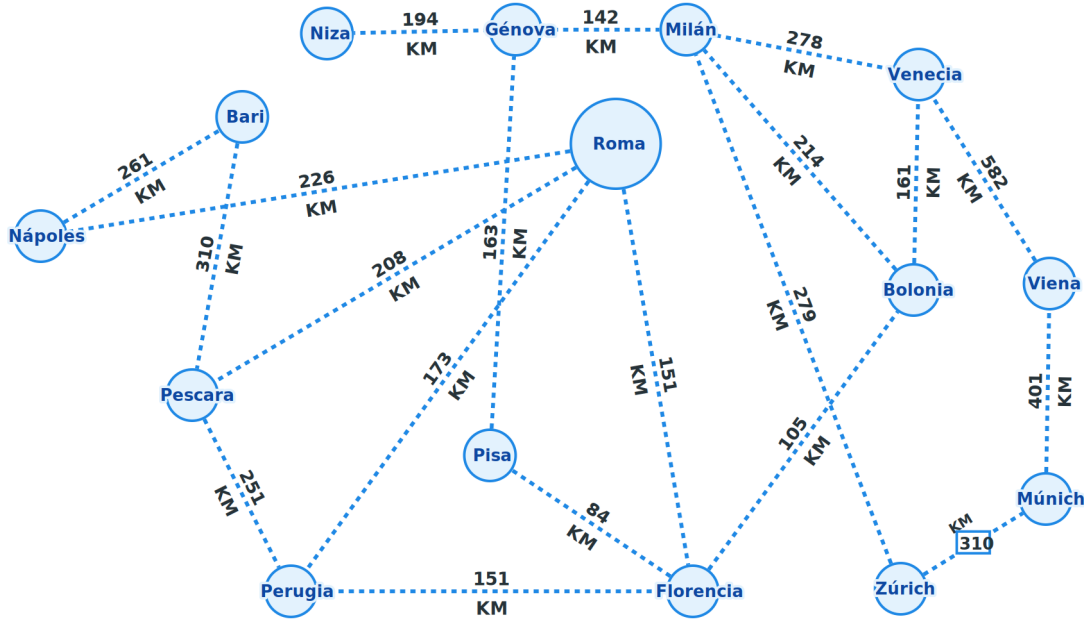


Figure 1: Grafo ponderado $G = (V, E, w)$ con las 15 ciudades y 20 conexiones. Los nodos representan ciudades y las aristas indican conexiones terrestres con su distancia en kilómetros.

2.4. Costo de un Camino

Dado un camino $P = (v_0, v_1, v_2, \dots, v_k)$, su costo total se define como:

$$C(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

2.5. Ruta Óptima

La ruta óptima entre una ciudad origen $s \in V$ y un destino $t \in V$ corresponde al camino P^* de costo mínimo:

$$P^* = \arg \min_P C(P)$$

considerando todos los caminos posibles que conectan s con t .

2.6. Propiedad de Conexidad

El grafo G es **conexo**: para cualquier par de vértices existe al menos un camino que los une.

$$\forall u, v \in V, \quad \exists P(u, v)$$

2.7. Propiedades Formales del Grafo

Las siguientes proposiciones describen las propiedades estructurales del grafo construido:

(P1) Positividad de los pesos:

$$\forall (u, v) \in E, \quad w(u, v) > 0$$

Todas las aristas representan distancias en kilómetros; por tanto, sus pesos son siempre positivos. Esta condición es necesaria para garantizar la correcta ejecución del algoritmo de Dijkstra.

(P2) Simetría (no dirigido):

$$\forall (u, v) \in E, \quad (v, u) \in E \wedge w(u, v) = w(v, u)$$

Las rutas entre ciudades son bidireccionales y poseen la misma distancia en ambos sentidos.

(P3) Optimalidad:

$$P^* \text{ es óptima } \iff \nexists P' \text{ tal que } C(P') < C(P^*)$$

La ruta obtenida es óptima si y solo si no existe ningún camino alternativo de menor costo.

(P4) Grado mínimo:

$$\forall v \in V, \quad \deg(v) \geq 1$$

Todo vértice tiene al menos una arista, garantizando que ninguna ciudad queda aislada.

3. Metodología

3.1. Algoritmo Seleccionado

Para resolver el problema de camino mínimo se seleccionó el **algoritmo de Dijkstra**. Este algoritmo encuentra la distancia mínima desde un nodo origen hacia todos los demás nodos de un grafo cuyos pesos son estrictamente positivos.

La elección resulta adecuada porque todas las aristas del grafo representan distancias terrestres en kilómetros, satisfaciendo la condición $w(u, v) > 0$ (Propiedad P1). Con $|V| = 15$ y $|E| = 20$, la complejidad del algoritmo con cola de prioridad es $O((|V| + |E|) \log |V|) \approx O(35 \cdot \log 15) \approx O(140)$ operaciones, lo que lo hace prácticamente instantáneo para este grafo.

3.2. Funcionamiento General

El algoritmo ejecuta los siguientes pasos:

1. Inicializar todas las distancias en ∞ .
2. Asignar distancia 0 al nodo de origen.
3. Seleccionar el nodo no visitado con menor distancia conocida.
4. Actualizar las distancias de sus vecinos si se encuentra una ruta más corta.
5. Repetir hasta haber procesado todos los nodos.

3.3. Pseudocódigo

```

1  ALGORITMO: Dijkstra(G, s, t)
2  ENTRADA:  G = (V, E, w): grafo ponderado,  $w(u,v) > 0$ 
3             s : vertice de origen
4             t : vertice de destino
5  SALIDA:   P* : camino de costo minimo desde s hasta t
6             C(P*) : costo total del camino
7
8  INICIO
9      // 1. Inicializacion
10     PARA CADA vertice v EN V HACER
11         dist[v] <- INFINITO
12         previo[v] <- NULO
13     FIN PARA
14     dist[s] <- 0
15     Q <- cola de prioridad con todos los vertices de V
16
17     // 2. Bucle principal
18     MIENTRAS Q no este vacio HACER
19         u <- vertice en Q con menor dist[u]
20         QUITAR u de Q
21         SI u == t ENTONCES ROMPER
22
23         PARA CADA vecino v de u, (u,v) en E HACER
24             alternativa <- dist[u] + w(u, v)
25             SI alternativa < dist[v] ENTONCES
26                 dist[v] <- alternativa
27                 previo[v] <- u
28             FIN SI
29         FIN PARA
30     FIN MIENTRAS
31
32     // 3. Reconstruccion del camino
33     P* <- lista vacia
34     actual <- t
35     MIENTRAS actual != NULO HACER
36         INSERTAR actual al inicio de P*
37         actual <- previo[actual]
38     FIN MIENTRAS
39
40     DEVOLVER P*, dist[t]
41 FIN ALGORITMO

```

Listing 1: Algoritmo de Dijkstra – Pseudocódigo

3.4. Comparación con Algoritmo Alternativo: Bellman-Ford

Table 2: Comparación entre Dijkstra y Bellman-Ford

Criterio	Dijkstra	Bellman-Ford
Complejidad temporal	$O((V + E) \log V)$ con cola de prioridad	$O(V \times E)$
Pesos negativos	No admite	Admite
Detección de ciclos negativos	No detecta	Sí detecta
Estrategia	Voraz (<i>greedy</i>): explora siempre el vértice de menor distancia acumulada	Relaja todas las aristas $ V - 1$ veces
Eficiencia en grafos grandes	Alta	Baja: más lento en grafos con muchas aristas
Aplicación en este proyecto	SELECCIONADO: todos los pesos son positivos; más eficiente	DESCARTADO: no aporta ventaja, ya que no existen pesos negativos

4. Implementación

4.1. Estructura General de la Aplicación

La aplicación fue desarrollada en **Python** y organizada en cinco módulos:

`datos.py` Define las 20 conexiones del grafo y expone la función `construir_grafo()`, que construye el diccionario de adyacencia bidireccional.

`Algoritmo.py` Contiene las funciones `dijkstra()`, `obtener_ruta()` y `ruta_minima()`, que implementan el cálculo del camino mínimo.

`mapa.py` Genera la representación visual del grafo utilizando `networkx` y `matplotlib`, destacando la ruta óptima calculada.

`interfaz.py` Construye la interfaz gráfica con `customtkinter`, gestiona la selección de ciudades y muestra los resultados al usuario.

`mapa_base.py` Administra el renderizado del mapa base sobre el que se superpone la visualización del grafo.

4.2. Representación del Grafo

El grafo se almacena como un **diccionario de adyacencia**, donde cada ciudad es una clave y sus conexiones son un diccionario secundario de vecino-peso:

```

1 {
2     "Roma": {
3         "Napoles": 226,
4         "Perugia": 173,
5         "Pescara": 208,
6         "Florenzia": 275

```



```

7     },
8     "Napoles": {
9         "Roma": 226,
10        "Bari": 261
11    },
12    ...
13 }

```

Listing 2: Fragmento del diccionario de adyacencia generado por `construir_grafo()`

Las conexiones se agregan en ambos sentidos para garantizar la bidireccionalidad:

$$\text{grafo}[\text{origen}][\text{destino}] = d \quad \wedge \quad \text{grafo}[\text{destino}][\text{origen}] = d$$

Esto asegura que el grafo sea no dirigido (Propiedad P2).

4.3. Implementación del Algoritmo de Dijkstra

```

1 import heapq
2
3 def dijkstra(grafo, origen):
4     distancias = {nodo: float("inf") for nodo in grafo}
5     anteriores = {nodo: None for nodo in grafo}
6     distancias[origen] = 0
7     cola_prioridad = [(0, origen)]
8
9     while cola_prioridad:
10         distancia_actual, ciudad_actual = heapq.heappop(
11             cola_prioridad)
12
13         if distancia_actual > distancias[ciudad_actual]:
14             continue
15
16         for vecino, peso in grafo[ciudad_actual].items():
17             nueva_distancia = distancias[ciudad_actual] + peso
18             if nueva_distancia < distancias[vecino]:
19                 distancias[vecino] = nueva_distancia
20                 anteriores[vecino] = ciudad_actual
21                 heapq.heappush(cola_prioridad, (
22                     nueva_distancia, vecino))
23
24     return distancias, anteriores
25
26 def obtener_ruta(anteriores, origen, destino):
27     ruta = []
28     actual = destino
29     while actual is not None:
30         ruta.append(actual)
31         actual = anteriores[actual]
32     ruta.reverse()

```

```

32     if ruta and ruta[0] == origen:
33         return ruta
34     return []
35
36
37 def ruta_minima(grafo, origen, destino):
38     distancias, anteriores = dijkstra(grafo, origen)
39     ruta = obtener_ruta(anteriores, origen, destino)
40     return ruta, distancias[destino]

```

Listing 3: Implementación de Dijkstra en Python (Algoritmo.py)

La función `dijkstra()` inicializa todas las distancias en ∞ y utiliza `heapq` como cola de prioridad mínima. La condición de la línea 10 evita procesar un nodo con una distancia ya desactualizada, optimizando el rendimiento. La función `obtener_ruta()` reconstruye el camino óptimo recorriendo el diccionario `anteriores` desde el destino hasta el origen.

4.4. Librerías Utilizadas

Table 3: Librerías utilizadas en el proyecto

Librería	Uso
<code>heapq</code>	Cola de prioridad para el algoritmo de Dijkstra
<code>customtkinter</code>	Interfaz gráfica moderna
<code>networkx</code>	Construcción y análisis del grafo
<code>matplotlib</code>	Visualización gráfica del grafo y la ruta

4.5. Interfaz Gráfica

La interfaz desarrollada con `customtkinter` permite:

- Seleccionar la ciudad de origen mediante un menú desplegable.
- Seleccionar la ciudad de destino mediante un menú desplegable.
- Ejecutar el cálculo de la ruta óptima presionando el botón *Calcular ruta*.
- Visualizar la secuencia de ciudades que componen el recorrido.
- Mostrar el costo total estimado en kilómetros.
- Visualizar el grafo completo con la ruta óptima destacada en color.

5. Resultados

Se realizaron tres pruebas de funcionamiento de la aplicación para validar la correcta ejecución del algoritmo. En cada prueba se indica la ciudad de origen, la ciudad de destino, la ruta óptima obtenida y el costo total estimado.

Table 4: Resumen de las tres pruebas de funcionamiento

N°	Origen	Ruta óptima obtenida	Costo (km)
1	Roma	Roma → Florencia → Bolonia → Milán → Zúrich → Múnich	1 183
2	Viena	Viena → Venecia → Bolonia → Florencia → Perugia → Pescara → Bari	1 560
3	Niza	Niza → Génova → Pisa → Florencia → Roma	886

5.1. Prueba 1: Roma → Múnich

Ruta óptima: Roma → Florencia → Bolonia → Milán → Zúrich → Múnich

Desglose del costo:

$$275 + 105 + 214 + 279 + 310 = 1\,183 \text{ km}$$

Esta ruta atraviesa el eje central de Italia de sur a norte hasta llegar a Milán, desde donde se conecta con Zúrich y finalmente Múnich. El algoritmo descarta la ruta alternativa vía Venecia–Viena–Múnich, cuyo costo sería:

$$275 + 105 + 161 + 582 + 401 = 1\,524 \text{ km}$$

confirmando que la ruta seleccionada es efectivamente la óptima.

5.2. Prueba 2: Viena → Bari

Ruta óptima: Viena → Venecia → Bolonia → Florencia → Perugia → Pescara → Bari

Desglose del costo:

$$582 + 161 + 105 + 151 + 251 + 310 = 1\,560 \text{ km}$$

Esta prueba demuestra la capacidad del algoritmo para encontrar rutas óptimas que involucren múltiples ciudades intermedias, optimizando la distancia total recorrida.

5.3. Prueba 3: Niza → Roma

Ruta óptima: Niza → Génova → Pisa → Florencia → Roma

Desglose del costo:

$$194 + 163 + 84 + 275 = 716 \text{ km}$$

Esta prueba valida el recorrido costero desde el sur de Francia hacia la capital italiana, mostrando que el algoritmo prioriza la ruta directa por la costa ligure.

6. Conclusiones

El desarrollo de este proyecto permitió aplicar conceptos fundamentales de teoría de grafos, tales como vértices, aristas, pesos, caminos y conectividad, en un contexto real de optimización de redes de transporte europeas.

La implementación del algoritmo de Dijkstra demostró ser eficiente para grafos ponderados con pesos positivos, obteniendo rutas óptimas de forma rápida y precisa. Su comparación con Bellman-Ford evidenció que, dado que todas las distancias del grafo son positivas, Dijkstra es la elección más adecuada tanto en precisión como en rendimiento computacional.

El desarrollo de la interfaz gráfica con `customtkinter` y la visualización del grafo con `networkx` y `matplotlib` facilitaron la interacción con el usuario y permitieron representar visualmente la estructura del grafo y los recorridos calculados.

Dificultades enfrentadas: La principal dificultad fue la recopilación y verificación de las distancias terrestres reales entre ciudades, dado que en algunos tramos las rutas disponibles varían significativamente según el tipo de vía utilizada. Además, la configuración del entorno gráfico con `customtkinter` requirió ajustes de compatibilidad entre versiones de la librería.

Mejoras futuras: Se considera incorporar un mapa geográfico real e interactivo, aumentar la cantidad de ciudades consideradas, utilizar tiempos reales de viaje en lugar de distancias estáticas, e implementar algoritmos adicionales como A^* para comparar rendimientos en grafos de mayor escala.

Referencias

Google Maps. (2026). *Distancias por carretera entre ciudades europeas*. Google LLC. Recuperado en junio de 2026 de <https://maps.google.com>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Diestel, R. (2017). *Graph Theory* (5th ed.). Springer. <https://doi.org/10.1007/978-3-662-53622-3>

NetworkX Developers. (2024). *NetworkX: Network Analysis in Python*. Recuperado de <https://networkx.org>

Hagberg, A., Swart, P., & Chult, D. (2008). *Exploring network structure, dynamics, and function using NetworkX*. Proceedings of the 7th Python in Science Conference (SciPy2008), 11–15.